

It Is Not the Signature: An Empirical Study of a Hidden Key-Parse Cost in JSON Web Token Validation

Jannatul Ferdous Katha^{a,c}, Tasmia Tahmid Prova^{b,c}, Md Kishor Morol^c, Tze Hui Liew^{d,*}, Dip Nandi^{c,e}

^a*Department of CSE, Bangladesh University of Professionals (BUP), Dhaka, Bangladesh*

^b*Department of CSE, Bangladesh University of Business and Technology (BUBT), Dhaka, Bangladesh*

^c*ELITE Research Lab, Queens, NY, USA*

^d*Faculty of Information Science and Technology, Multimedia University, Melaka, Malaysia*

^e*Department of Computer Science, American International University-Bangladesh (AIUB), Dhaka, Bangladesh*

Abstract

Developers routinely attribute the per-request cost of JSON Web Token (JWT) validation to cryptography. We test that assumption in `jsonwebtoken`, a widely used Node.js library. The signature primitive is 2.208 μ s of a 29.92 μ s validation; 18.04 μ s (60%) is a discarded parse, in which a string secret is first offered to the asymmetric key parser and is resolved only after that parse fails. Its price is set by a transitive dependency, the OpenSSL release beneath the runtime: issued from C, the same calls cost 394.4 μ s on OpenSSL 3.0.16 and 8.447 μ s on 3.5.8. The slow case is not confined to end-of-life runtimes: Ubuntu 24.04's packaged Node.js, linked to the system OpenSSL 3.0, pays 436 μ s per validation. In a containerised service on OpenSSL 3.0, validation accounted for 53% of the application's processor time per request. Of seven JWT libraries in four languages, only this one uses a failing parse, and only its penalty changes by an order of magnitude with the OpenSSL release. The obvious optimisation is unsafe: the parse arrived with a key-confusion security fix, and removing it naively fails the library's own tests. A narrow fix passes the full suite and makes validation 4.15 times faster; only a pre-parsed

*Corresponding author.

Email addresses: `ferdous.katha@elitelab.ai` (Jannatul Ferdous Katha), `tasmia.prova@elitelab.ai` (Tasmia Tahmid Prova), `kishormorol@ieee.org` (Md Kishor Morol), `thliew@mmu.edu.my` (Tze Hui Liew), `dip.nandi@aiub.edu` (Dip Nandi)

key removes the cost for every input, yet none of the 327 call sites in our sample pre-parses it. We derive guidance for developers, library authors and performance engineers, and release a replication package.

Keywords: Empirical study, performance analysis, software libraries, root-cause analysis, JSON Web Token, OpenSSL

1. Introduction

Performance engineering depends on attributing cost correctly. When a cost is believed to belong to one component and belongs to another, optimisation effort goes to the wrong place, and architectural remedies are adopted for problems that could have been fixed in a few lines. Empirical studies of performance bugs find that many stem from inefficient library usage and avoidable redundant work, and that the fixes are small once the cause is located (Jin et al., 2012; Selakovic and Pradel, 2016). This paper reports such a case on a path that runs on every request of many web services: the validation of JSON Web Tokens (JWTs) (Jones et al., 2015).

The usual reasoning is intuitive: validating a token means verifying a signature, signatures are cryptography, and cryptography is expensive, so the cost should be *moved* to a sidecar proxy, a gateway or dedicated hardware. The assumption has been tested before one layer down, where Coarfa et al. (2006) found RSA a minority of TLS server cost. We test it at the application layer, in `jsonwebtoken`, a widely used Node.js library, and find it wrong. A second natural explanation, that key material is converted from a string into a key object on every call, fails as well: the conversion costs 0.583 μ s.

The actual mechanism is a control-flow accident (Figure 2). Given a secret that is not already a key object, the library attempts `createPublicKey()` first and falls back to `createSecretKey()` in the exception handler. For an HMAC secret the first call cannot succeed, so every validation constructs and discards an OpenSSL parse failure, at 18.04 μ s of a 29.92 μ s call. We call this failed `createPublicKey()` call, including the exception that reports its failure, the *discarded parse*. Its price depends on which OpenSSL release performs it, by more than an order of magnitude: runtimes carrying OpenSSL 3.2 or later pay tens of microseconds, runtimes carrying the 3.0 series pay hundreds. The 3.0 series is still the system OpenSSL of long-term-support distributions, and Node.js as packaged by Ubuntu 24.04 links it and pays the slow price (Section 5.3); so does any image pinned to an older base layer, which is

31 common in practice (Zerouali et al., 2019).

32 For software engineering the case matters for three reasons beyond the
33 library itself. First, the cost sits in control flow that no profiler label names:
34 a parse whose failure is caught and discarded appears in profiles as OpenSSL
35 work, and the natural reading of that attribution is “cryptography”. Second,
36 its magnitude is set by a transitive dependency that application developers do
37 not choose directly and rarely record. Third, the obvious fix is unsafe. The
38 resolution block entered the library in version 9.0.0, the release that fixed an
39 RSA-to-HMAC key-confusion vulnerability (Auth0, 2022), and the key-type
40 checks implementing that fix read the type of the key object it produces;
41 removing it the obvious way fails the library’s own key-confusion tests. The
42 code that costs the time is the code that makes a security check work, and the
43 coupling is not visible from the lines being changed. The design that produces
44 the cost (resolving key material on every call, by way of a parse whose failure
45 serves as a type test) can also be named independently of `jsonwebtoken`,
46 and it predicts which of seven libraries perform a failing parse (Section 7).

47 *Research questions.*

48 **RQ1** Where does the per-request cost of token validation actually go?

49 **RQ2** What governs the magnitude of that cost?

50 **RQ3** Does it matter in a running containerised service, and how common is
51 the pattern that avoids it?

52 **RQ4** Is the defect a property of one library, or of a design choice other
53 libraries could share?

54 **RQ5** Can the cost be removed without weakening the library’s key-confusion
55 defence, and who can cause it to be paid?

56 *Contributions.* All are supported by measurements in the replication pack-
57 age (Katha et al., 2026).

- 58 1. Two independent decompositions of the validation call that locate its
59 dominant component in a discarded parse, and a split of it into the
60 OpenSSL parse, the V8 exception and the binding (Section 4).
- 61 2. A causal attribution of its magnitude to the bundled OpenSSL release,
62 with the parse reissued from C against 8 releases (Section 5).
- 63 3. Corroboration in a containerised service by cgroup CPU accounting, and
64 a survey showing the avoiding pattern is rare in public code (Section 6).
- 65 4. A comparison of seven JWT libraries in four languages showing that
66 the cost follows a design choice, not JWT validation as such, and a

- 67 detector-based survey of 64 plus 164 popular npm packages locating
68 the failing-parse form of that choice (Section 7).
- 69 5. An analysis of why the cost cannot be removed naively, and a narrow fix
70 that passes the library’s full test suite and is 4.15 times faster (Section 8).
- 71 6. Guidance for developers, library authors and performance engineers,
72 including the one-line change that removes the cost (Section 9).

73 2. Background and related work

74 *What a validation call does.* A JSON Web Token carries a header, a payload
75 and a signature over the first two (Jones et al., 2015). Validating one means
76 decoding two segments, resolving key material into a form the cryptographic
77 library accepts, and verifying the signature. The middle step, key resolution,
78 is the subject of this paper, and it is the step most often left implicit when
79 validation cost is discussed.

80 *Where the cost is usually attributed.* Measurement of layered enforcement in
81 microservices concentrates on transport and proxy layers; Zhu et al. (2023)
82 show that service-mesh sidecar overhead depends on configuration and work-
83 load rather than being a fixed tax. Work on cryptographic library performance
84 has largely stayed at the level of the primitive or the protocol exchange (Naser
85 et al., 2020; Sikeridis et al., 2020). Our result concerns neither: it is the cost
86 of getting key material *into* the library, which we could not find measured
87 directly.

88 *Performance bugs and anti-patterns.* Jin et al. (2012) found that most real-
89 world performance bugs follow a few recurring patterns, among them skippable
90 function calls, and that fixes are usually local; Nistor et al. (2013) found
91 that most are discovered by reasoning about code rather than by observing
92 symptoms. In JavaScript, Selakovic and Pradel (2016) identify inefficient API
93 usage as a leading root cause, and their later profiler (Selakovic et al., 2017)
94 targets the *order* in which a program evaluates conditions, the axis on which
95 this defect lives. The defect here is a skippable call whose failure is its only
96 observable effect.

97 *Exceptions as control flow.* Exceptions are costly in managed runtimes, mostly
98 in their construction (Ogasawara et al., 2006), and Parravicini and Mueller
99 (2021) provide the closest cost-attribution baseline for V8, though on specula-
100 tion rather than exceptions. Studies of how exceptions are used find handlers

101 that discard what they catch to be common (Cabral and Marques, 2007; Kery
102 et al., 2016; de Pádua and Shang, 2017). That literature treats a swallowed
103 exception as a maintainability smell; here it is control flow on a hot path,
104 and Section 4.1 measures how much of its cost is the exception itself.

105 *Dependencies and the npm ecosystem.* Performance changes across versions in
106 ways maintainers do not anticipate (Kaltenecker et al., 2023; Chen and Shang,
107 2017). Container images pin a dependency set implicitly (Cito et al., 2017)
108 and routinely ship outdated packages (Zerouali et al., 2019). In the npm
109 ecosystem, Staicu and Pradel (2018) show that a slow path inside widely used
110 modules reaches the web servers that depend on them, and Davis et al. (2018)
111 that on Node.js’s single event loop one slow synchronous handler delays every
112 other client. For OpenSSL 3, maintainers of a major downstream library
113 have stated that parsing and key loading regressed against 1.1.1 (Gaynor and
114 Kehrer, 2026); Section 5.2 supplies a controlled measurement for the one path
115 this paper concerns.

116 *Benchmarking pitfalls and change detection.* Our protocol follows established
117 guidance for managed-runtime benchmarking (Georges et al., 2007; Kalibera
118 and Jones, 2013; Barrett et al., 2017). Microbenchmarks go wrong in recurring
119 ways (Costa et al., 2021), developers misjudge when warm-up ends (Traini
120 et al., 2023), cloud environments degrade their reliability (Laaber et al., 2019),
121 and benchmark suites often fail to detect performance changes at all (Laaber
122 and Leitner, 2018). The attribution this paper overturns has the structure
123 Mytkowicz et al. (2009) warn about: a second variable, the bundled OpenSSL
124 version, moving unrecorded alongside the one under study.

125 *Security and performance.* Security mechanisms are routinely evaluated for
126 their performance cost: Coarfa et al. (2006) for TLS servers, and Sikeridis
127 et al. (2020) for post-quantum TLS and SSH. In that line the cost is the
128 price of the protection. Here the cost and the protection are coupled in a less
129 visible way: the code that costs the time is the code a security check depends
130 on (Section 8.3).

131 *Cryptographic APIs and JWT libraries.* Most cryptographic vulnerabilities
132 are misuses of libraries by applications (Lazar et al., 2014; Egele et al., 2013),
133 largely because the interfaces are hard to use (Nadi et al., 2016); detection has
134 been automated by specifying correct usage (Krüger et al., 2018; Rahaman
135 et al., 2019). Our finding differs: the interface admits a use that is *correct*

and costly only for a reason invisible at the call site. Security analyses of JWT implementations consistently find key-handling defects, key confusion among them (Detering et al., 2017; Xu et al., 2023); Yang et al. (2026) evaluated 43 JWT libraries across 16 languages. RFC 8725 (Sheffer et al., 2020) codifies the resulting practice. The library studied here has been hardened against key confusion twice: CVE-2015-9235, fixed in 4.2.2 (GitHub Advisory Database, 2015), and CVE-2022-23541 with two related defects, fixed in 9.0.0 (Auth0, 2022). We treat the same surface as a cost site and show the two views are coupled. Our survey follows guidance on mining public repositories (Kalliamvakou et al., 2014).

2.1. What we searched for and did not find

Table 1 summarises the positioning. Each cell records only what this section states about the work cited; it is a map of the lines of work this paper connects, not an assessment of them.

We found no peer-reviewed study of performance regressions across the OpenSSL 3.x series, none tracing the chain from base image to bundled library to application performance, and none treating a correct-but-costly use of a cryptographic interface as a performance defect. We state each as an absence we searched for.

3. Methodology

Figure 1 gives the study’s workflow: the instruments used for each research question and the tables and figures each produces. Every instrument writes to committed run files, from which every number in the paper is generated (Data availability).

3.1. Testbed

Unless stated otherwise, measurements were taken on one Apple-silicon machine running macOS 26.5.2 (arm64), *the host*, under Node.js v26.6.0. This Node.js is the Homebrew build, which links the system’s Homebrew OpenSSL dynamically rather than bundling its own; the host therefore ran OpenSSL 3.6.3, newer than the 3.5 series that official Node 26 builds bundle. The version was not recorded per row for the host decomposition, and is inferred from the run record (Section 10). Container measurements ran in the Linux/arm64 virtual machine of Docker Desktop’s single-node Kubernetes cluster (v1.32.2). The service studied in Section 6 is a Node.js microservice

Table 1: Positioning against the closest lines of work. *JWT*: studies JWT or JOSE implementations. *Crypto-API*: treats how a cryptographic interface or key material is used as a security concern. *Cost*: studies a performance cost or performance defect. *Version*: attributes a cost or defect to the version of a bundled dependency. *Coupling*: shows that removing a cost would weaken a security defence. *Breadth*: studies many systems, projects or libraries, which this paper does not (one library in depth, six others for comparison). ✓: addressed; ◦: partly (see note); -: not addressed.

Work	JWT	Crypto-API	Cost	Version	Coupling	Breadth
Detering et al. (2017); Xu et al. (2023); Yang et al. (2026)	✓	✓	–	–	–	✓
Egele et al. (2013); Nadi et al. (2016); Krüger et al. (2018); Rahaman et al. (2019)	–	✓	–	–	–	✓
Coarfa et al. (2006); Naser et al. (2020); Sikeridis et al. (2020)	–	–	✓	–	–	–
Zhu et al. (2023)	–	–	✓	–	–	–
Jin et al. (2012); Selakovic and Pradel (2016)	–	–	✓	–	–	✓
Kaltenecker et al. (2023); Chen and Shang (2017)	–	–	✓	◦ ^a	–	✓
Zerouali et al. (2019); Cito et al. (2017)	–	–	–	◦ ^b	–	✓
This paper	✓	✓	✓	✓	✓	◦ ^c

^aVersions of the system under study, not of a bundled dependency. ^bContainer images pin outdated dependency versions; no cost is measured. ^cOne library in depth; seven for comparison; a detector run over a stated frame of npm packages (Section 7).

170 (Express 5.1.0, `jsonwebtoken` 9.0.3) on the `node:18-alpine` image, run as
171 one replica with an Istio sidecar behind an NGINX ingress, verifying HS256
172 tokens with a shared secret supplied as a string. Load was generated in open
173 loop by k6 v2.2.0 on the same machine, so every in-situ window records the
174 host load average observed during it. This is a controlled single-node testbed,
175 not a production deployment; we call it the *containerised service*.

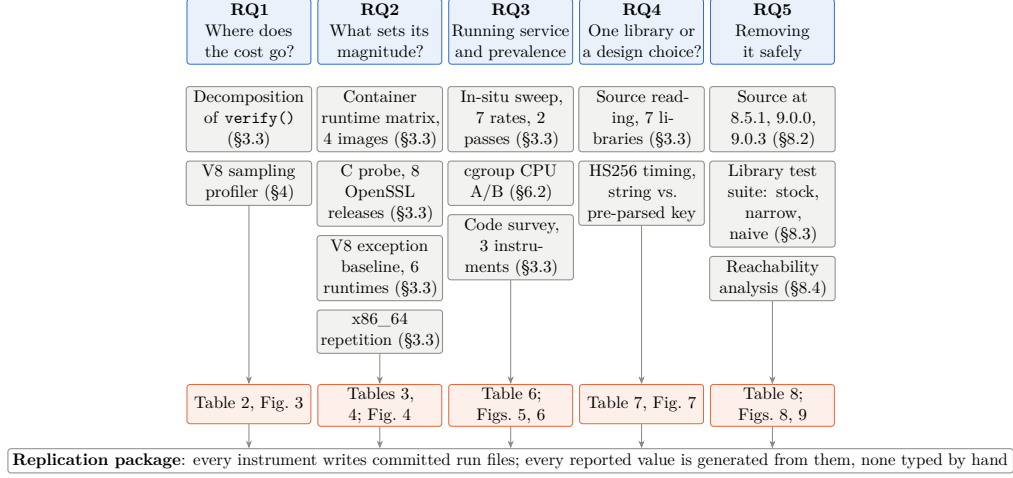


Figure 1: Study workflow. Top: the five research questions. Middle: the instruments that answer each, with the section describing it. Bottom: the tables and figures each produces. The C probe and the exception baseline (RQ2) also decompose the discarded parse for RQ1 (Table 3), and the call-site survey (RQ3) supplies the algorithm-pinning figure used in RQ5. All measurements feed one replication package, from which every reported value is generated.

176 3.2. Measurement protocol

177 The unit of measurement is one operating-system process, not one call.
 178 Each process performs a fixed number of timed iterations of a single condition
 179 after a warmup of half that number and contributes its median; reported
 180 figures are medians of per-process medians, with 95% percentile bootstrap
 181 intervals over processes (10,000 resamples) (Georges et al., 2007). With 15
 182 processes per condition these intervals are indicative rather than calibrated;
 183 the per-process values are in the replication package, and its analysis script
 184 can also report BCa intervals. Conditions are interleaved within each round,
 185 and differences between conditions are reported as the median of per-round
 186 *paired* differences. The largest between-process coefficient of variation is 3.5%
 187 on the host.

188 3.3. Instruments

189 *Decomposing the call.* `jwt.verify()` of `jsonwebtoken` version 9.0.3 is de-
 190 composed into the operations it performs, each timed in isolation: structural
 191 decode, the HMAC primitive with a string and with a pre-parsed key, the
 192 symmetric key constructor, and the asymmetric key parse the library attempts

```

120 if (secretOrPublicKey != null && !(secretOrPublicKey instanceof
    KeyObject)) {
121   try {
122     secretOrPublicKey = createPublicKey(secretOrPublicKey);
123   } catch (_) {
124     try {
125       secretOrPublicKey = createSecretKey(typeof
        secretOrPublicKey === 'string' ? Buffer.from(
        secretOrPublicKey) : secretOrPublicKey);
126     } catch (_) {
127       return done(new JsonWebTokenError('secretOrPublicKey is not
        valid key material'))
128     }
129   }
130 }

```

Figure 2: Key resolution in `jsonwebtoken` version 9.0.3, `verify.js`, lines 120–130. Verbatim but for the common leading indentation, which is removed to fit the column; line numbers are the file’s own. A secret that is not already a `KeyObject` is offered to `createPublicKey()` first, and `createSecretKey()` is reached only from the exception handler. For an HMAC secret supplied as a string the outer `try` can never succeed, so the asymmetric parse is attempted and discarded on every validation.

193 first (Figure 2), both when it throws and when it succeeds. Each condition
 194 runs in 15 processes of 40000 calls.

195 *Runtime matrix.* Containerisation is held constant and the runtime varied:
 196 four container images spanning two OpenSSL series and three V8 major
 197 versions, on one machine, at 10 processes of 20000 calls. The runtime identity
 198 of each image is recorded with every row and cross-checked (0 mismatches).

199 *C probe.* To remove the JavaScript engine, we reproduced in C, call for call
 200 from the Node source, what `createPublicKey()` asks of OpenSSL for an
 201 HMAC secret (three PEM scans, a private-key parse and the error-queue
 202 drain), compiled with one compiler (GCC 13.3.0) against 8 OpenSSL releases
 203 from 3.0.16 to 3.6.1 built identically from source, at 15 processes per release.

204 *Exception baseline.* An `Error` of the same message, properties and code as
 205 Node’s (captured from a real failure) is thrown from 8 frames and discarded,
 206 with no OpenSSL call, on six official Node runtimes. With the C probe this

207 gives a three-way decomposition of the discarded parse: the OpenSSL parse,
208 the V8 exception, and a remainder attributed to the binding between them.

209 *Other libraries.* We read the key-handling code of six further libraries,
210 the most used in their ecosystems (`fast-jwt`, `jose`, `PyJWT`, `golang-jwt`,
211 `nimbus-jose-jwt`, `jjwt`), and timed HS256 verification in each with the
212 secret as a string (or the rawest accepted form) and as the library’s own
213 pre-parsed key object, at 15 processes of 20000 calls.

214 *Second architecture.* The host decomposition, the runtime matrix, the C
215 probe, the exception baseline and the cross-library comparison were measured
216 on an x86_64 virtual machine (Intel(R) Xeon(R) Processor @ 2.80GHz, four
217 vCPUs) on shared cloud infrastructure. Following Laaber et al. (2019), we
218 take only orderings and ratios between interleaved conditions from it. Two
219 Node 22 builds appear in these runs: the repetition of the host decomposition
220 used that machine’s own Node 22.22.2 (OpenSSL 3.5.5), while the exception
221 baseline used the official Node 22.23.3 release binary, which bundles OpenSSL
222 3.5.8.

223 *In-situ measurement.* The containerised service exposes a histogram timing
224 the verification call alone, read before and after each of seven arrival-rate
225 steps; means are exact differences of `_sum` over `_count`. The sweep ran twice
226 on one process, ascending then descending. An A/B at one rate, validation
227 enabled and then disabled, reads the processor time charged to the container
228 from cgroup accounting.

229 *Prevalence.* Three instruments with different frames, captured on 2026-09-18:
230 code-search co-occurrence counts, a classified convenience sample of call sites
231 (one file per repository, key argument traced within its file), and a targeted
232 counter-search that fetches every file co-occurring the library with the pre-
233 parsing interface and adjudicates it by hand. None supports a population
234 proportion, so results are reported *in our sample*.

235 *Distribution-packaged Node.js.* To test whether the OpenSSL 3.0 case survives
236 on supported systems, an Ubuntu 24.04 image with the distribution’s own
237 `nodejs` package, which links the system OpenSSL, is measured beside two
238 official images under the runtime-matrix protocol (Section 5.3).

239 *Rejected tokens.* A further harness times `verify()` on well-formed tokens
 240 that the client can build without the secret, one declaring RS256 and one
 241 declaring HS256 with a signature not made with the secret, under the stock
 242 library, the narrow fix and a pre-parsed key, and a second script compares
 243 the stock library and the narrow fix on every combination of key material,
 244 token and algorithm option (Section 8.4).

245 *Library survey.* To move RQ4 from existence towards prevalence, a frame
 246 of npm packages is drawn from the registry’s search interface with stated
 247 queries and a weekly-download threshold, and a detector built on a JavaScript
 248 parser flags every `try/catch` whose `try` calls a key or certificate parser and
 249 whose handler falls back to a different key constructor. Every hit is read and
 250 adjudicated by hand (Section 7.1).

251 4. RQ1: Where the cost goes

Table 2: Cost of each operation in the validation path. Medians of per-process medians over 15 processes of 40000 calls; 95% percentile bootstrap intervals over processes, which with 15 units are indicative rather than calibrated (Section 3.2).

Operation	Median (μ s)	95% interval (μ s)
<code>verify()</code> , string secret	29.92	[29.75, 30.12]
discarded asym. parse	18.04	[17.83, 18.25]
successful asym. parse	13.83	[13.62, 13.92]
<code>verify()</code> , pre-parsed key	6.042	[5.958, 6.125]
HMAC, string key	2.208	[2.208, 2.250]
HMAC, pre-parsed key	2.166	[2.125, 2.167]
structural decode	1.292	[1.250, 1.292]
symmetric key constructor	0.583	[0.542, 0.583]
timer overhead	0.042	[0.042, 0.042]

252 Table 2 and Figure 3 give each operation in the validation path. The
 253 signature primitive costs 2.208 μ s of a 29.92 μ s call. The conversion to which
 254 the remainder is most readily attributed, the symmetric key constructor, costs
 255 0.583 μ s. The discarded parse that precedes it costs 18.04 μ s, 60% of the call
 256 and its largest single component. Supplying a pre-parsed key instead of a

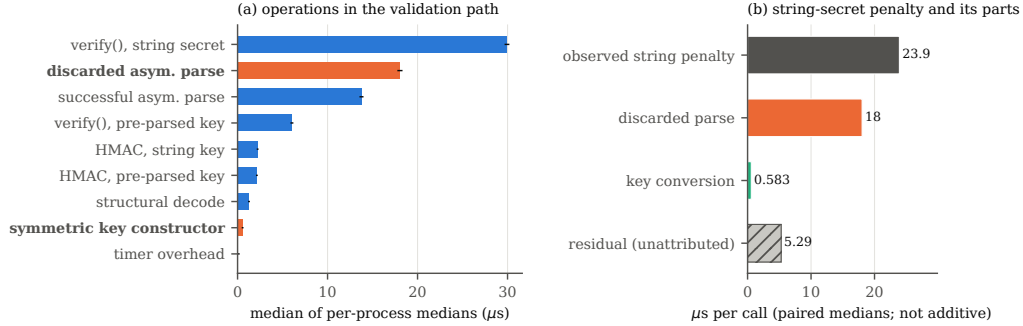


Figure 3: (a) Every operation in Table 2, on a linear scale, with the two components the paper turns on emphasised; whiskers are the 95% percentile bootstrap intervals over 15 processes. (b) The 23.92 μs penalty for supplying a string secret rather than a pre-parsed key, beside the discarded parse, the key conversion and the unattributed residual. Each bar is a median of per-round paired differences or of per-process medians, not part of one measurement, so the bars are not additive; the 5.292 μs residual is the part of the penalty the two measured components leave unexplained (Section 4.2).

257 string reduces the call by 23.92 μs [23.58, 24.00]. The asymmetry often read
 258 as conversion being dearer than parsing is backwards: a *successful* asymmetric
 259 parse costs 13.83 μs , so the RSA path’s parse is useful work while the HMAC
 260 path’s is waste.

261 4.1. What the discarded parse is made of

262 Table 3 decomposes the discarded parse on six official runtimes. On every
 263 runtime bundling OpenSSL 3.0 the C parse alone matches the probe to within
 264 a few percent (101% of it under Node 18.20.8): the discarded parse *is* the
 265 OpenSSL parse and would cost the same with no JavaScript engine. The V8
 266 exception is small and nearly constant, about 4.781 μs (1.2% of the probe
 267 under Node 18.20.8), half of it stack capture. Where OpenSSL 3.5 makes the
 268 parse cheap, the three parts are comparable: under Node 26.10.0 the parse
 269 is 47%, the exception 19.0% and the binding 34.1%. The three-way split is
 270 interpretable only for these OpenSSL 3.5 rows. On the OpenSSL 3.0 rows the
 271 binding comes out negative (-8.694 μs under Node 18.20.8), because our stock
 272 C build of each release and Node’s own build of it differ by more than the
 273 binding costs; there only the dominance of the parse is meaningful. Either
 274 way the component is a discarded *parse*, not an exception cost.

Table 3: The discarded parse decomposed, per runtime, in μs (x86_64). *Probe*: `createPublicKey(secret)` inside Node, thrown and caught. *C parse*: the same OpenSSL calls issued from C against the OpenSSL release that runtime bundles, built from source. *V8 exception*: an `Error` of the same shape, message and properties thrown from 8 frames and discarded, with no OpenSSL call. *Binding*: the probe less the other two; its interval, from independent resampling of the three runs, is in the replication package. Every column is a median of per-process medians over 15 processes. The C column uses a stock build of each release, not Node’s own build of it, so the binding column is approximate: it is negative on two OpenSSL 3.0 rows, and the three-way split is interpretable only for the OpenSSL 3.5 rows (Section 4.1).

Runtime	OpenSSL	Probe	C parse	V8 exception	Binding
Node 18.20.8	3.0.16	390.5	394.4	4.781	-8.694
Node 20.20.2	3.0.19	443.4	431.3	4.796	7.292
Node 23.11.1	3.0.16	383.5	394.4	3.959	-14.88
Node 22.23.3	3.5.8	18.98	8.447	4.021	6.515
Node 24.21.0	3.5.8	19.54	8.447	4.136	6.952
Node 26.10.0	3.5.8	18.01	8.447	3.421	6.145

275 4.2. Checks on the decomposition

276 The discarded parse and the conversion account for 77.9% of the string
277 penalty. The 5.292 μs residual [4.916, 5.501] presumably lies in the library’s
278 own type and claim checking, but it was not measured directly and is reported
279 as unattributed. Because the decomposition relies on differencing, it was
280 checked against V8’s sampling profiler: the asymmetric key parse accounts
281 for a median 66.2% of self time on the host (95.3% under Node 18 and 61.5%
282 under Node 26), and does not appear among the top frames of the pre-parsed
283 path. The two methods agree on which component dominates, not on its
284 exact share: differencing gives 60% and the profiler 66.2% on the host. They
285 measure different quantities. The differenced share divides the median of
286 the parse, timed alone in its own processes, by the median of the whole call;
287 the profiler’s share attributes sampled time inside one validation loop to the
288 innermost frame, so work the parse frame triggers (including the construction
289 of its exception) is counted with it, and sampling carries attribution error of
290 its own (Mytkowicz et al., 2010). We therefore rely on the ordering, not on
291 either percentage.

Table 4: The same measurements across four container runtimes on one machine (arm64), 10 processes of 20000 calls each, in μs . *Discarded parse*: `createPublicKey()` on the HMAC secret, which fails. *Successful parse*: `createPublicKey()` on an RSA-2048 public key. *Pre-parsed*: `verify()` with a pre-parsed key. Across the four images the discarded parse spans $25.36\times$, the successful parse $11.74\times$ and pre-parsed validation $1.26\times$. [‡]From Table 2; not containerised. [†]The Homebrew OpenSSL the host’s Node links, inferred from the run record rather than recorded per row (Section 3.1).

Runtime	OpenSSL	V8	Discarded parse	Successful parse	Pre-parsed <code>verify()</code>
Node 18 musl	3.0.16	10	373.79	115.83	7.58
Node 20 musl	3.0.19	11	420.62	142.38	7.25
Node 26 musl	3.5.7	14	22.94	15.54	6.56
Node 26 glibc	3.5.7	14	16.58	12.12	6.04
host [‡]	3.6.3 [†]	14	18.04	13.83	6.042

292 5. RQ2: What governs the magnitude

293 5.1. What the runtime matrix shows

294 Table 4 varies the runtime with containerisation held constant. The dis-
 295 carded parse spans $25.36\times$ across the four images, while pre-parsed validation
 296 spans only $1.26\times$. Node 18 and Node 20 ship different V8 major versions
 297 and share the OpenSSL 3.0 series; both pay of order 373.79–420.62 μs , so
 298 within that series the engine does not move the cost. A successful parse moves
 299 with the failing one ($11.74\times$), so the effect concerns key parsing in general.
 300 The matrix alone cannot separate OpenSSL from the engine across the series
 301 boundary, since the fast images also carry a newer engine and a different
 302 build; the crossed designs below do.

303 5.2. Caused by OpenSSL

304 In plain C, with nothing else varying, the failure costs 394.4 μs on OpenSSL
 305 3.0.16 and 8.447 μs on 3.5.8, a ratio of 46.69 [46.16, 46.90] (Figure 4). The
 306 magnitude is therefore *caused* by the OpenSSL release, for this code path.
 307 Almost all of it is the private-key parse (99% on 3.0.16), where OpenSSL 3
 308 hands the input to its provider-based decoders. The cost fell in two steps, to
 309 57.48 μs in 3.1.8 and to 6.998 μs in 3.2.6, and is flat thereafter. The engine
 310 is excluded from the other direction too: Node 23.11.1 pairs a *newer* V8
 311 with the *older* OpenSSL 3.0.16 and pays 383.5 μs , against 18.98 μs for Node
 312 22.23.3 with OpenSSL 3.5.8.

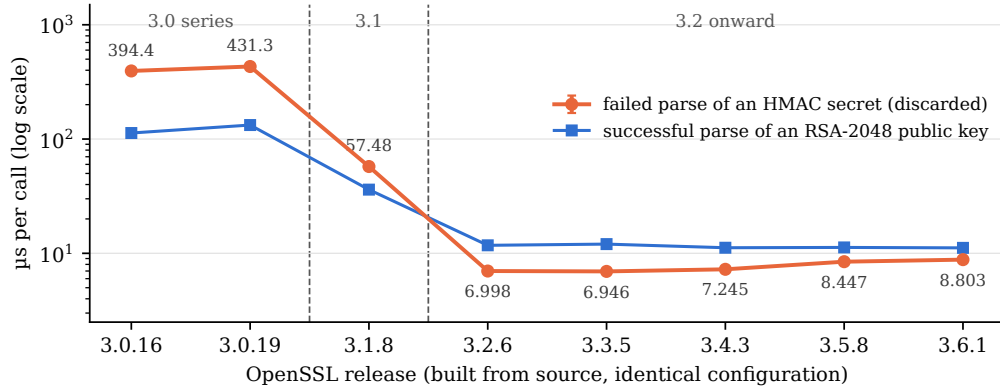


Figure 4: The failed parse of an HMAC secret issued from C, with no JavaScript engine, against 8 OpenSSL releases built from source with one configuration, beside a successful parse of an RSA-2048 public key through the same path. Whiskers are 95% percentile bootstrap intervals over 15 processes and are narrower than the markers. The cost falls in two steps, at 3.1 and 3.2, and is flat thereafter.

313 *A second architecture.* On x86_64 every OpenSSL 3.0 image is again slower
 314 than every OpenSSL 3.5 image (the split holds): the discarded parse spans
 315 $30.50\times$ across the images while pre-parsed validation spans $1.31\times$. The host
 316 decomposition keeps its ordering: the discarded parse is $19.42\ \mu\text{s}$ of a $36.48\ \mu\text{s}$
 317 call (53%).

318 5.3. Distribution-packaged Node.js

319 The OpenSSL 3.0 series is not confined to end-of-life Node.js images.
 320 Ubuntu 24.04 LTS packages Node.js 18.19.1, built against the system’s shared
 321 OpenSSL rather than a bundled copy; in its current image that is OpenSSL
 322 3.0.13. Table 5 measures it beside two of the official images of Table 4, on
 323 one x86_64 virtual machine (4 vCPUs) in one run (2026-09-30), with the
 324 protocol of Section 3.2.

325 On Ubuntu 24.04’s own Node.js the discarded parse costs $417\ \mu\text{s}$, the same
 326 order as on the end-of-life `node:18` image measured alongside it ($634\ \mu\text{s}$) and
 327 an order of magnitude above the OpenSSL 3.5 image ($39.3\ \mu\text{s}$). Supplying the
 328 secret as a string costs $436\ \mu\text{s}$ per validation [433, 441] against a pre-parsed
 329 key, and the narrow fix brings the call to $12.2\ \mu\text{s}$. The OpenSSL 3.0 price is
 330 therefore not an artefact of end-of-life images: a service that runs the Node.js
 331 its LTS distribution ships pays it today. How many deployments do so we
 332 cannot say; we have no data on the runtimes `jsonwebtoken` users deploy.

Table 5: Distribution-packaged Node.js against official images, same x86_64 machine and run, 10 processes of 20,000 calls per condition (one process of the `node:18` pre-parsed cell failed without output), in μs (medians of per-process medians). *Penalty*: median of per-round paired differences between `verify()` with a string secret and with a pre-parsed key, with its 95% bootstrap interval. Absolute values are not comparable with the arm64 values of Table 4.

Runtime	OpenSSL	Discarded parse	Successful parse	<code>verify()</code> , string	<code>verify()</code> , pre-parsed	Penalty	95% interval
Ubuntu 24.04, Node 18.19.1	3.0.13 (system)	417	123	446	9.74	436	[433, 441]
<code>node:18.20.8-alpine</code>	3.0.16 (bundled)	634	192	685	10.2	675	[668, 680]
<code>node:26.6.0-alpine</code>	3.5.7 (bundled)	39.3	19.2	58.8	8.30	50.6	[49.7, 51.3]

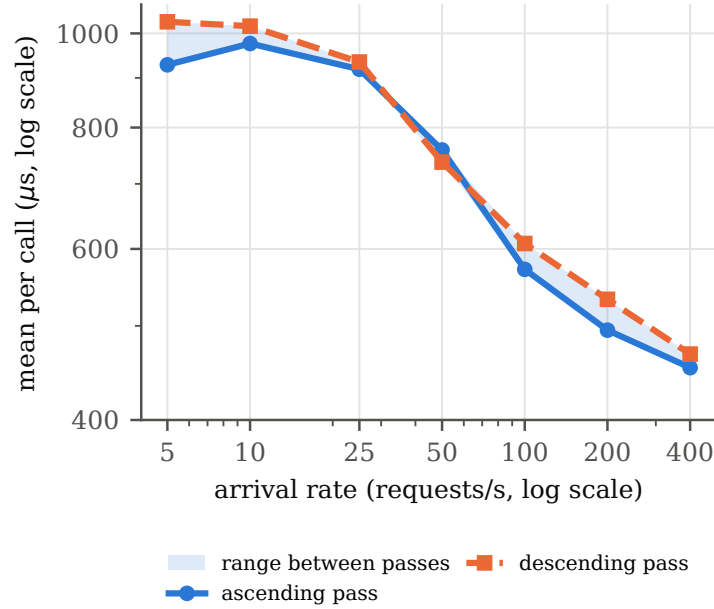


Figure 5: In-situ validation cost against arrival rate, string secret. The two passes are drawn separately rather than averaged, because they differ systematically and averaging would conceal that. The shaded band is the range between the passes, which is two observations rather than an interval.

333 6. RQ3: The containerised service, and prevalence

334 6.1. In-situ cost

335 Figure 5 shows the in-situ cost at seven arrival rates. The cost is present
 336 in the containerised service and far larger than the host figure, consistent with
 337 Section 5: the service’s runtime carries the older OpenSSL, whose discarded

338 parse costs 373.79 μ s, giving an expected total near 381 μ s. Wall-clock means
 339 of 453–1028 μ s exceed that; contention inflates durations, so they are upper
 340 bounds. Two features of Figure 5 are not explained by validation and we
 341 draw no conclusion from them. The cost falls as the arrival rate rises, and
 342 the descending pass differs from the ascending one systematically rather
 343 than randomly. Plausible causes, none of which we isolated, are a fixed
 344 per-window overhead amortised over more calls at higher rates; processor
 345 frequency scaling and idle-state exit latency, which penalise sparse arrivals;
 346 cache and branch-predictor residency, which improve with call density; and
 347 continued JIT tier-up and garbage-collection timing, which differ between a
 348 process’s first and second pass. The magnitude claim rests instead on the
 349 processor-time measurement below.

350 6.2. Corroboration: the cost is processor time

Table 6: Validation enabled against validation disabled, at one arrival rate, two 180 s
 windows on the same pod and image, 36001 requests each. A signed token is on the wire in
 both arms; the disabled arm removes only the handler’s `jwt.verify()` call. Application
 and sidecar CPU are means of one-second cgroup samples in the window; CPU per call is
 that mean divided by the achieved rate. Event-loop lag is the mean of the per-scrape p99.
 Host load is a single sample at window start.

	disabled	enabled
Achieved rate (req/s)	200.006	200.006
Application CPU (millicores)	69.45	146.38
CPU per call (μs)	347.3	731.9
Sidecar CPU (millicores)	53.40	49.13
Event-loop lag (ms)	3.137	3.159
Client mean latency (ms)	1.6802	2.0332
Host load, 1 min	3.53	5.11

351 To separate work from waiting, Table 6 compares the same service with
 352 validation enabled and disabled, reading processor time from cgroup account-
 353 ing rather than from a clock. Enabling validation raised the application’s
 354 processor time per request from 347.3 to 731.9 μ s: validation accounted for
 355 53% of the application’s CPU per request, a factor of 2.11 over the service
 356 without it. The added 384.6 μ s is of the same size as the 381 μ s the runtime
 357 matrix predicts for the same work on the same runtime, from instruments that
 358 share no code. A request that queues accrues wall-clock time but no CPU,

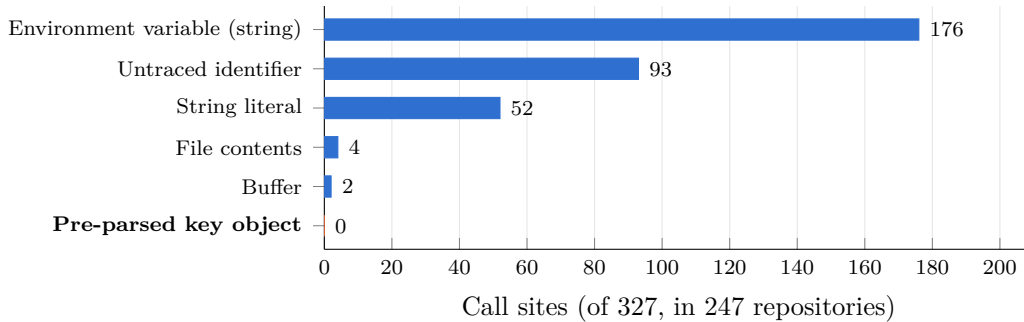


Figure 6: Where the secret passed to the validator comes from, for the 327 call sites classified in our sample. No call site passes a pre-parsed key object, the one form that avoids the discarded parse for every input. Independently, 309 of the 327 call sites (94%) pass no algorithm restriction. Counts describe the sample, not the population (Section 3.3).

so this figure excludes queueing delay; it can still be inflated by contention for caches, SMT siblings and processor frequency on the shared host. The sidecar’s CPU did not rise with the application’s, and the one-second CPU samples of the two arms do not overlap (disabled never exceeded 100.577 millicores, enabled never fell below 114.963). This is one pair of consecutive windows at different host load, so the difference carries no interval; a repeated, alternating design is prepared for the authors’ testbed (Section 10).

6.3. How common is the avoiding pattern, in our sample?

In code-search counts, no cell exceeds 0.0303% of files importing the library that also mention the pre-parsing interface. Of 327 call sites classified across 247 repositories, 0 pass a pre-parsed key (Figure 6), and 309 (94%) pass no algorithm restriction. Call sites are found by resolving the local binding of the import, which adds 12 that the conventional `jwt.verify()` spelling misses. The targeted counter-search fetched 191 files, 120 with a validation call in 112 repositories. Of the 13 call sites it flagged automatically, hand inspection found 5 false positives and 1 that could not be resolved within its file; 7 were genuine, in 6 projects. Even among repositories that know the interface exists, only 5.36% use it: the pattern is rare, not absent. The zero in the classified sample may be biased downwards: keys are traced within one file only, and 93 of the 327 call sites pass an identifier whose origin lies in another file and could, in principle, be a key object built there. No file in the sample mentions

Table 7: HS256 verification across seven JWT libraries in four languages (x86_64), in μ s. *Asym. first?*: given a string HMAC secret, does the library attempt to parse it as an asymmetric key, as read from its source. *String* and *pre-parsed*: the secret in the form the library’s documentation shows, and in the library’s own key object built once (Section 3.3). *Penalty*: median of per-round paired differences, with a 95% bootstrap interval over rounds. Medians of per-process medians over 15 processes of 20000 calls. Absolute costs measure different amounts of work in different runtimes and are not a ranking of libraries. ^aNo string accepted for an HMAC key; the rawest accepted form (bytes) is used. ^bA string is accepted only by a deprecated setter that resolves it once, when the parser is built.

Library	Lang.	Version	Crypto backend	Asym. first?	String	Pre-parsed	Penalty	95% interval
jsonwebtoken	JS	9.0.3	OpenSSL 3.0.19	yes	476.2	10.23	465.9	[465.2, 467.8]
jsonwebtoken	JS	9.0.3	OpenSSL 3.5.8	yes	34.22	9.844	24.56	[24.07, 24.82]
fast-jwt	JS	6.3.3	OpenSSL 3.0.19	no	7.564	7.570	0.070	[-0.116, 0.111]
fast-jwt	JS	6.3.3	OpenSSL 3.5.8	no	7.917	7.934	-0.068	[-0.134, 0.197]
jose	JS	6.2.12	OpenSSL 3.0.19	no ^a	121.2	78.42	42.66	[38.50, 45.89]
jose	JS	6.2.12	OpenSSL 3.5.8	no ^a	98.89	73.98	24.80	[21.91, 26.51]
PyJWT	Python	2.15.0	hashlib, OpenSSL 3.0.13	no	41.33	41.28	-0.103	[-0.457, 0.338]
golang-jwt	Go	5.3.1	Go crypto/hmac	no ^a	4.810	4.764	0.034	[-0.042, 0.117]
nimbus-jose-jwt	Java	10.10	JCA SunJCE	no	4.358	4.229	0.304	[-0.098, 1.016]
jjwt	Java	0.13.0	JCA SunJCE	no ^b	12.15	12.33	-0.406	[-0.715, 0.648]

the pre-parsing interface in any spelling, which weighs against that, but it is not proof.

7. RQ4: One library, or a design choice?

A cost found in one library becomes a statement about a class of defect only if its mechanism can be named independently and predicts which other libraries have it. The mechanism here is *resolving key material of unknown type by attempting the asymmetric parse first, and treating its failure as the signal that the material is a shared secret*. Table 7 and Figure 7 test that prediction.

Only `jsonwebtoken` attempts the asymmetric parse. The others avoid it in one of three ways: *inspect, then dispatch* (`fast-jwt` and `PyJWT` examine the key text and call exactly one constructor); *refuse the ambiguous input* (`jose` and `golang-jwt` do not accept a string HMAC key); or *type the key* (`jjwt` requires a `SecretKey`; `nimbus-jose-jwt` converts a string’s bytes without parsing them).

Of the seven libraries, two pay for a raw secret, in the sense that the paired interval on the penalty excludes zero; the other five pay nothing measurable. The two are `jsonwebtoken` and `jose`, and they pay for different reasons. `jose` parses nothing that can fail, but imports raw bytes into a `WebCrypto` key on

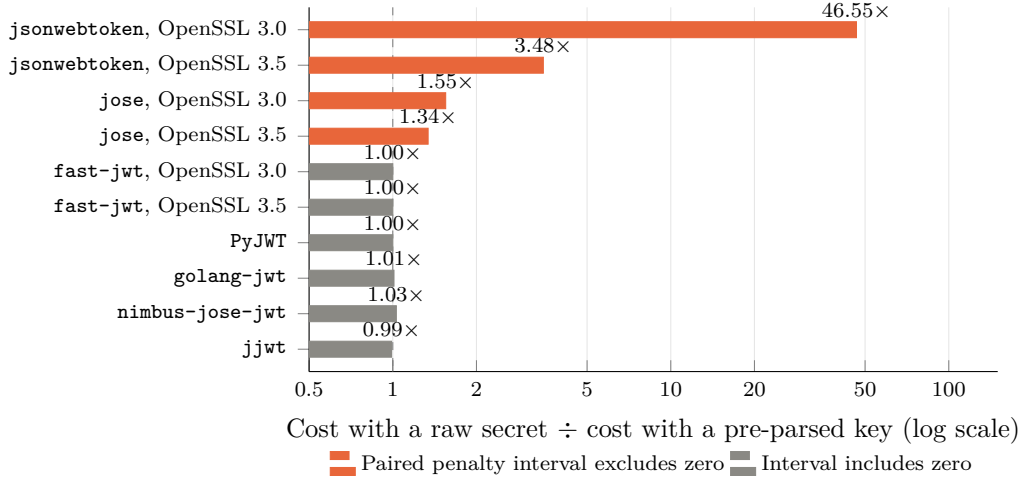


Figure 7: How much each library pays for being handed its raw secret rather than its own pre-parsed key object: the ratio of the two costs in Table 7, on a log scale. A ratio of one (dashed line) means no per-call penalty. Orange: the paired 95% interval on the penalty excludes zero. `jsonwebtoken` and `jose` both pay for per-call key handling; under OpenSSL 3.5.8 their absolute penalties are indistinguishable, and `jsonwebtoken`’s larger ratio there reflects `jose`’s higher baseline. Only `jsonwebtoken` performs a failing asymmetric parse, and only its penalty changes by an order of magnitude with the OpenSSL release.

every call. Under OpenSSL 3.5.8 the two absolute penalties are statistically indistinguishable: $24.56 \mu\text{s}$ [24.07, 24.82] for `jsonwebtoken` and $24.80 \mu\text{s}$ [21.91, 26.51] for `jose`. `jsonwebtoken`’s larger ratio in Figure 7 there reflects `jose`’s higher baseline, not a larger penalty. The libraries separate under OpenSSL 3.0.19: `jsonwebtoken`’s penalty rises to $465.9 \mu\text{s}$ and `jose`’s to $42.66 \mu\text{s}$, about 11 times less. The distinguishing property of `jsonwebtoken` is therefore not that it alone pays for per-call key handling, but that it does so through a *failing* parse, whose price follows the OpenSSL release by an order of magnitude. All seven libraries validate the same kind of token with the same primitive, so the cost follows a design choice, resolving key material on every call, and not JWT validation as such. The failing parse is the form of that choice most exposed to the cryptographic library underneath, and the class of order-of-evaluation defect the profiler of Selakovic et al. (2017) targets.

413 *7.1. How often is a failing parse used as a type test?*

414 Seven libraries show that the design choice exists; they do not show how
415 often it is made. We therefore searched for it systematically in one ecosystem
416 (Section 3.3). The first frame holds the npm packages returned for six JWT
417 and JOSE keyword queries with at least 50,000 weekly downloads (64 of
418 1727 returned, captured 2026-09-30); the second, wider frame holds packages
419 returned for nine cryptography, signature, certificate and authentication
420 queries with at least 500,000 weekly downloads (164 of 6222). The detector
421 parsed all 11767 and 9989 JavaScript files of the published tarballs, with 0
422 and 0 failures.

423 It flagged 1 package in the first frame and 0 in the second. The one
424 package is `jsonwebtoken` itself, at 2 sites, both confirmed by reading and
425 both on a per-call path: the `verify()` block studied here, and the same shape
426 in `sign()`, which offers the configured secret to `createPrivateKey()` before
427 falling back to `createSecretKey()`. Signing therefore pays a discarded parse too;
428 we did not time it. Within these frames, then, the failing parse used as
429 a type test is not a common idiom but a property of one library, whose reach
430 comes from that library’s own adoption. The detector matches one syntactic
431 shape, so a type test written differently (for example by inspecting an error
432 code, or through a third-party parser) is not flagged, and the count is a lower
433 bound within the frames; other ecosystems remain to be surveyed.

434 **8. RQ5: Removing the cost safely**

435 *8.1. Threat model, and the discarded parse precedes authentication*

436 The adversary is a remote client that can place arbitrary bytes where the
437 service expects a bearer token, including every header field, but does not
438 know the shared secret. The asset is key-type separation: a token declaring
439 an HMAC algorithm must never be verified against asymmetric key material,
440 nor the reverse (Sheffer et al., 2020). We measure what a rejected forged
441 token costs per call (Section 8.4), but we drove no adversarial traffic at a
442 service and make no claim about availability or capacity. Figure 8 shows
443 the order of checks in `verify()`. Reaching key resolution requires only a
444 well-formed token; the permitted-algorithm list, the key-type checks and the
445 signature all come later. A token that will be rejected therefore pays the
446 discarded parse in full, and restricting permitted algorithms does not avoid it.

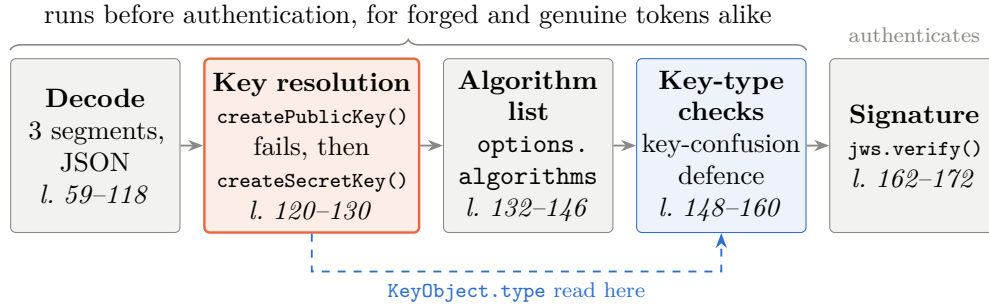


Figure 8: Order of operations in `verify()` at tag v9.0.3, with the file’s line numbers. Every step up to and including the algorithm and key-type checks runs before the signature is verified, and so runs for a forged token exactly as for a genuine one. The discarded parse sits inside key resolution; the key-type checks that implement the library’s key-confusion defence read the type of the key object resolution produces (dashed arrow). Pinning permitted algorithms is enforced at the third step and therefore does not prevent the second.

447 8.2. The discarded parse arrived with a key-confusion fix

448 Comparing `verify.js` in versions 8.5.1, 9.0.0 and 9.0.3: in 8.5.1 the
 449 secret is passed to the signature routine as given, and the resolution block of
 450 Figure 2 first appears in 9.0.0, the release that fixed CVE-2022-23541 and
 451 CVE-2022-23539 (Auth0, 2022). The checks implementing those fixes (lines
 452 148–160) operate on the key object the block produces. The discarded parse
 453 is part of a security fix, not an oversight.

454 8.3. A fix, and a constraint on it

455 The fix dispatches on the token’s declared algorithm before attempting
 456 the asymmetric parse, so an HMAC token with an ordinary string secret is
 457 resolved directly. It takes the call from 29.92 μ s to 7.208 μ s, a factor of 4.15,
 458 and on x86_64 from 36.48 μ s to 11.09 μ s (Figure 9), with no change in calling
 459 code. Against the library’s own suite at tag v9.0.3 the patch leaves results
 460 unchanged: 511 passing, 0 failing.

461 The constraint is the more important half. The *success* of the parse
 462 on asymmetric key material produces the key object whose type the key-
 463 confusion check depends on. The fix must therefore be narrow: the fast
 464 path is restricted to string secrets that cannot be asymmetric key material.
 465 Removing that restriction, the obvious optimisation, produces 2 failures in
 466 the library’s key-confusion tests (“should not allow HMAC verification with

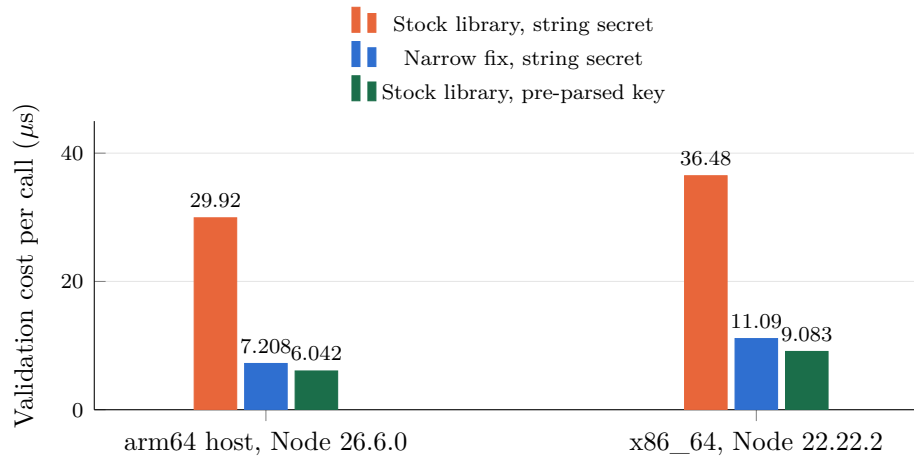


Figure 9: Per-call HS256 validation cost under the stock library with a string secret, under the narrow fix with the same string secret, and under the stock library with a pre-parsed key, on both machines (medians of per-process medians). The narrow fix recovers most of the gap without any change to calling code; only the pre-parsed key removes the discarded parse for every input (Section 8.4).

Table 8: Which tokens reach the discarded parse, for a service configured with an HMAC shared secret, as read from the source at tag v9.0.3 and from the patch of Section 8.3. “Well-formed”: three segments, JSON header and payload, non-empty signature. No row requires knowledge of the secret.

Configuration	Tokens that reach the parse
Stock, string secret	every well-formed token, forged or genuine
Narrow fix, plain string secret (no PEM or JWK form)	every well-formed token whose header declares a non-HMAC algorithm; no such token can be valid for this service
Pre-parsed <code>KeyObject</code>	none

467 an RSA key in PEM format” and, under “signing with pub key as symmetric”,
 468 “should not verify”). The existing suite already guards the property; what is
 469 missing is visibility of the coupling to someone changing the code.

470 8.4. What the narrow fix leaves reachable

471 The fast path is selected by the declared algorithm, which the token’s
 472 author chooses. A token declaring a non-HMAC algorithm falls through to
 473 the original resolution and pays the parse before being rejected (Table 8).

Table 9: Cost of rejecting a forged token, in μ s per `verify()` call (medians of per-process medians), for a service configured with an HMAC shared secret and `algorithms: ['HS256']`. *RS256 header*: a token declaring RS256 with a random signature, rejected by the algorithm list. *HS256 header*: a token declaring HS256 with a signature not made with the secret, rejected at signature verification. *Narrow*: the narrow fix of Section 8.3.

Runtime	OpenSSL	RS256 header			HS256 header		
		Stock	Narrow	Pre-parsed	Stock	Narrow	Pre-parsed
node:18.20.8-alpine	3.0.16	693	673	19.2	720	30.9	27.0
Ubuntu 24.04, Node 18.19.1	3.0.13	462	463	21.1	482	32.2	28.4
node:26.6.0-alpine	3.5.7	112	113	58.3	78.2	25.5	22.7
Node 22.22.2 (VM)	3.5.5	35.5	35.6	14.2	40.8	21.4	19.2

474 The narrow fix removes the cost from legitimate traffic, not from traffic an
 475 adversary constructs. Only a pre-parsed key object removes it for every input,
 476 because resolution is skipped when the key is already a `KeyObject`.

477 *Measured.* Table 9 times the rejection of well-formed forged tokens, which
 478 the client can produce without the secret, under each configuration
 479 (`bench/forged-token.js`; 10 processes of 10000 calls per cell, on an x86_64
 480 virtual machine, Intel(R) Xeon(R) Processor @ 2.10GHz). The measurements
 481 confirm the source reading. A token declaring RS256 is rejected by the
 482 algorithm list, yet under both the stock library and the narrow fix it first
 483 pays the discarded parse: on OpenSSL 3.0.16 693 μ s and 673 μ s per rejection,
 484 and on Ubuntu 24.04’s OpenSSL 3.0.13 462 μ s and 463 μ s, against 19.2 μ s
 485 and 21.1 μ s with a pre-parsed key. A forged token declaring HS256, by
 486 contrast, is cheap to reject under the narrow fix (30.9 μ s against 720 μ s stock
 487 on OpenSSL 3.0.16). The remainder in the pre-parsed column is the rejection
 488 itself, which varies by runtime. These are per-call costs in a microbenchmark;
 489 we did not drive such tokens at a running service, and draw no conclusion
 490 about availability.

491 *Behaviour under misconfiguration.* The narrow fix takes its fast path only for
 492 a string that carries neither a PEM header nor a JSON object, so a PEM or
 493 JWK key mistakenly configured as the HMAC “secret” resolves exactly as it
 494 does today. We checked this rather than assuming it: for 9 kinds of configured
 495 key material (including PEM public and private keys and a JWK string
 496 supplied as the secret), 252 combinations of key, token and algorithm option
 497 were verified through the stock library and the narrow fix on each of 4 runtimes,
 498 and 0 produced a different result or error (`bench/keypath-equivalence.js`).

```

const { createSecretKey } = require('node:crypto');
const jwt = require('jsonwebtoken');

// Once, at startup: build the key object.
const key = createSecretKey(Buffer.from(process.env.JWT_SECRET));

// Per request: pass the key object instead of the string.
jwt.verify(token, key, { algorithms: ['HS256'] });

```

Figure 10: The one-line change that removes the discarded parse for every input: the secret is converted to a `KeyObject` once, and `verify()` skips key resolution when handed one (Figure 2, line 120).

499 8.5. Reported upstream

500 An earlier issue¹ reports HS256 verification in 9.x 34 times slower than in
 501 8.5.1 on OpenSSL 3.0, with no diagnosed cause. We reported the mechanism²,
 502 and a pull request implementing the narrow fix was opened against it. At
 503 the time of this revision (30 September 2026) the pull request had not been
 504 merged, no release after 9.0.3 had been published, and the repository’s main
 505 branch had not changed since June 2026.

506 9. Implications for practice

507 Table 10 maps each finding to its evidence and to the stakeholder who
 508 can act on it.

509 *Application developers.* Pre-parse key material once, at startup (Figure 10).
 510 It removes 23.92 μ s per validation on the host and far more on OpenSSL 3.0,
 511 needs no architectural change and no change to the library, and is the only
 512 remedy that removes the discarded parse for every input. Also restrict
 513 permitted algorithms (Sheffer et al., 2020); that is an independent defence,
 514 not a substitute.

515 *Performance engineers.* Treat the base image as part of the measurement.
 516 The same code differs by 25.36 $\times$ on this path across images because they

¹<https://github.com/auth0/node-jwt-token/issues/966>

²<https://github.com/auth0/node-jwt-token/issues/1046>

Table 10: Summary of findings, evidence and implications, by research question and by the stakeholder who can act on each.

RQ	Finding	Evidence	Implication (who acts)
RQ1	The signature is 2.208 μ s of a 29.92 μ s call; a discarded parse is 18.04 μ s (60%).	Table 2, Fig. 3; profiler	Profile before relocating a “cryptographic” cost; attribution by intuition was wrong twice here (<i>performance engineers</i>).
RQ2	The parse costs 394.4 μ s on OpenSSL 3.0.16 and 8.447 μ s on 3.5.8, issued from C; Ubuntu 24.04’s own Node.js pays the OpenSSL 3.0 price.	Tables 4, 5; Fig. 4	Record the base image and bundled OpenSSL with every performance figure; treat base-image bumps as performance changes (<i>developers, DevOps</i>).
RQ3	In a containerised service on OpenSSL 3.0, validation is 53% of the application’s CPU per request; none of the 327 sampled call sites pre-parses its key.	Table 6; Figs. 5, 6	Pre-parse key material once at startup, a one-line change (Figure 10) (<i>application developers</i>).
RQ4	Of seven libraries in four languages only one uses a failing parse; two pay for per-call key handling, and only the failing parse’s cost changes by an order of magnitude with the OpenSSL release; in 64+164 popular npm packages only <code>jsonwebtoken</code> uses the failing parse.	Table 7, Fig. 7; §7.1	Do not use a failing parse as a type test; resolve keys once, inspect then dispatch, or type the key (<i>library authors</i>).
RQ5	The parse is load-bearing for a key-confusion defence; the naive optimisation fails 2 library tests, the narrow fix passes all 511 and is 4.15 $\times$ faster.	Table 8; Figs. 8, 9	Review performance changes to verification code as security changes; run the key-confusion tests (<i>maintainers, reviewers</i>).

517 pin different OpenSSL releases, and a regression after a base-image bump
518 may have nothing to do with the application. Profile before relocating a cost

519 believed to be cryptographic to a sidecar or gateway.

520 *Library authors.* Do not use a failing parse as a type test. Inspect and
521 dispatch, refuse ambiguous input, or type the key, and resolve keys once
522 rather than on every call.

523 *Maintainers and reviewers.* Review performance changes to verification code
524 as security changes, run them against the key-confusion tests, and time them
525 on the oldest cryptographic library a deployment may bundle.

526 10. Threats to validity

527 *Construct.* The decomposition relies on differencing, cross-checked by a pro-
528 filer whose share differs from the differenced one for the reasons given in
529 Section 4.2. The C probe reproduces Node’s OpenSSL calls but not Node’s
530 own build of OpenSSL, so the binding column of Table 3 is approximate;
531 the causal claim compares releases under one build and does not inherit this.
532 Every measurement uses HS256 with one secret of 34 characters and one
533 token of fixed size. The parse fails at the PEM-header check and in the
534 private-key decoder, and its cost does not depend on the secret’s length: a
535 16- and a 256-character secret cost 674 μ s and 675 μ s on OpenSSL 3.0.16
536 (Section 8.4). A secret that begins with a PEM header takes the original path
537 under the narrow fix by design and was not timed. The security analysis is
538 from source, the library’s tests and the microbenchmarks of Section 8.4, not
539 from adversarial traffic against a service.

540 *Internal.* The in-situ evidence is one process sweeping seven rates twice, and
541 the A/B is one pair of consecutive windows at different host load with the
542 load generator on the same machine. CPU accounting excludes queueing
543 delay, but contention for caches, SMT siblings and processor frequency can
544 still raise the CPU time charged for the same work, so the A/B difference is an
545 upper bound on the work and carries no interval. An alternating design with
546 the load generator pinned or remote is prepared in the replication package
547 but was not run. The headline host figure predates the per-measurement
548 OpenSSL field; its version (3.6.3) is inferred from the run record, and a re-run
549 with the version recorded is prepared.

550 *External.* Primary measurements come from one ARM64 host; the x86_64
551 machines are shared virtual machines, used only for orderings and ratios.
552 The containerised service is a single-node testbed on an end-of-life Node 18
553 image; the OpenSSL 3.0 behaviour it shows is also present in Ubuntu 24.04's
554 own Node.js (Section 5.3), but the service itself was not re-measured on a
555 supported image. A service that caches verified tokens pays the cost once
556 per token. Seven libraries at one version each show a design choice; the npm
557 survey of Section 7.1 estimates how often a failing parse is used as a type test
558 within a stated frame of one ecosystem, not across ecosystems.

559 *Reliability of the survey.* Call-site buckets are assigned by rules that trace a
560 key within one file, and the counter-search and the library-survey hits were
561 adjudicated by one person. A blinded, stratified sheet of call sites, a codebook
562 and an agreement script for a second rater are in the replication package; the
563 second rating has not yet been done.

564 *Future work.* Driving forged tokens at the running service, repeating the A/B
565 as alternating windows on supported images, bisecting OpenSSL between 3.0
566 and 3.2, and extending the library survey to PyPI, Go and Maven would each
567 close a limitation stated above.

568 11. Conclusion

569 The per-request cost of token validation in `jsonwebtoken` is not the
570 signature ($2.208\ \mu\text{s}$ of $29.92\ \mu\text{s}$) and not key conversion ($0.583\ \mu\text{s}$). It is a
571 parse attempted in the wrong order and discarded on every request, 60% of
572 the call. Its price is set causally by the OpenSSL release beneath the runtime:
573 $394.4\ \mu\text{s}$ on 3.0.16 against $8.447\ \mu\text{s}$ on 3.5.8, and the slow case persists in the
574 Node.js that Ubuntu 24.04 ships. Per-call key handling is a design choice, not
575 a property of JWT; `jsonwebtoken` is distinguished by making it through a
576 failing parse, whose cost changes by an order of magnitude with the OpenSSL
577 release. That parse cannot be removed naively, because it is load-bearing for
578 a key-confusion defence. A narrow fix preserves the defence and is 4.15 times
579 faster, and pre-parsing the key once is the only configuration no input can
580 push onto the slow path, yet no call site in our sample does so.

581 The broader lesson is about attribution. The explanation that presented
582 itself before measurement was wrong twice, and the component that sets
583 the cost is one that measurement protocols rarely record. Performance
584 studies should record the identity of the components they measure, including

transitive dependencies such as the bundled cryptographic library, at the granularity at which they can change.

CRedit authorship contribution statement

Jannatul Ferdous Katha: Conceptualization, Methodology, Software, Investigation, Data curation, Formal analysis, Visualization, Writing – original draft. **Tasmia Tahmid Prova:** Validation, Writing – review & editing. **Md Kishor Morol:** Supervision, Writing – review & editing. **Tze Hui Liew:** Supervision, Project administration, Writing – review & editing. **Dip Nandi:** Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

The data supporting the results of this study are openly available in the replication package (Katha et al., 2026), archived at Zenodo under <https://doi.org/10.5281/zenodo.23011153> and maintained in a public GitHub repository.³ It contains all measurements, code and scripts that generate every value and figure in this paper, and is released under the MIT licence. The third-party survey corpus is not redistributed; a manifest pins each of its 502 files by repository, commit, path and content hash, and a script in the package re-fetches every file from its repository and checks the hash, so the corpus can be reconstructed wherever the files remain public. Experiments that need the authors’ testbed, and the second rating of the survey, are scripted in the package with their exact commands.

³<https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominates-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing->

613 **Ethics and responsible disclosure**

614 The study involves no human participants and no personal data, and
615 no attack was run against any system the authors do not control. The key-
616 confusion behaviour discussed is a publicly documented attack class, exercised
617 only through the library’s own test suite.

618 **Declaration of generative AI and AI-assisted technologies in the** 619 **manuscript preparation process**

620 During the preparation of this manuscript, generative AI tools were used to
621 improve language and readability. The authors reviewed all content following
622 such use and take full responsibility for the contents of the manuscript.

623 **Acknowledgements**

624 The authors acknowledge ELITE Research Lab for providing the compu-
625 tational resources and support necessary for this research.

626 **References**

- 627 Auth0, 2022. CVE-2022-23539, CVE-2022-23541, CVE-2022-23540: Security update
628 for jsonwebtoken. Auth0 security bulletin, 21 December 2022, [https://auth0.](https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-jsonwebtoken)
629 [com/docs/secure/security-guidance/security-bulletins/2022-12-21-j](https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-jsonwebtoken)
630 [sonwebtoken](https://auth0.com/docs/secure/security-guidance/security-bulletins/2022-12-21-jsonwebtoken). Accessed 2026-09-28.
- 631 Barrett, E., Bolz-Tereick, C.F., Killick, R., Mount, S., Tratt, L., 2017. Virtual
632 machine warmup blows hot and cold. *Proceedings of the ACM on Programming*
633 *Languages* 1, 52:1–52:27. doi:10.1145/3133876.
- 634 Cabral, B., Marques, P., 2007. Exception handling: A field study in Java and
635 .NET, in: *ECOOP 2007 – Object-Oriented Programming*, Springer. pp. 151–175.
636 doi:10.1007/978-3-540-73589-2_8.
- 637 Chen, J., Shang, W., 2017. An exploratory study of performance regression
638 introducing code changes, in: *2017 IEEE International Conference on Software*
639 *Maintenance and Evolution (ICSME)*. doi:10.1109/ICSME.2017.13.
- 640 Cito, J., Schermann, G., Wittern, J.E., Leitner, P., Zumberi, S., Gall, H.C., 2017.
641 An empirical analysis of the Docker container ecosystem on GitHub, in: *2017*
642 *IEEE/ACM 14th International Conference on Mining Software Repositories*
643 *(MSR)*. doi:10.1109/MSR.2017.67.

644 Coarfa, C., Druschel, P., Wallach, D.S., 2006. Performance analysis of TLS web
645 servers. *ACM Transactions on Computer Systems* 24. doi:10.1145/1124153.11
646 24155.

647 Costa, D., Bezemer, C.P., Leitner, P., Andrzejak, A., 2021. What’s wrong with
648 my benchmark results? Studying bad practices in JMH benchmarks. *IEEE*
649 *Transactions on Software Engineering* 47, 1452–1467. doi:10.1109/TSE.2019.2
650 925345.

651 Davis, J.C., Williamson, E.R., Lee, D., 2018. A sense of time for JavaScript
652 and Node.js: First-class timeouts as a cure for event handler poisoning, in:
653 *Proceedings of the 27th USENIX Security Symposium*, pp. 343–359.

654 Detering, D., Somorovsky, J., Mainka, C., Mladenov, V., Schwenk, J., 2017. On
655 the (in-)security of JavaScript object signing and encryption, in: *Proceedings*
656 *of the 1st Reversing and Offensive-oriented Trends Symposium (ROOTS ’17)*.
657 doi:10.1145/3150376.3150379.

658 Egele, M., Brumley, D., Fratantonio, Y., Kruegel, C., 2013. An empirical study
659 of cryptographic misuse in Android applications, in: *Proceedings of the 2013*
660 *ACM SIGSAC Conference on Computer and Communications Security (CCS*
661 *’13)*. doi:10.1145/2508859.2516693.

662 Gaynor, A., Kehrer, P., 2026. The state of OpenSSL for pyca/cryptography.
663 pyca/cryptography project statement, <https://cryptography.io/en/latest/statements/state-of-openssl/>. Maintainer statement, not peer reviewed;
664 accessed 2026-09-24.

666 Georges, A., Buytaert, D., Eeckhout, L., 2007. Statistically rigorous Java perfor-
667 mance evaluation, in: *Proceedings of the 22nd Annual ACM SIGPLAN Confer-*
668 *ence on Object-Oriented Programming Systems, Languages and Applications*
669 *(OOPSLA ’07)*. doi:10.1145/1297027.1297033.

670 GitHub Advisory Database, 2015. Verification bypass in jsonwebtoken (CVE-2015-
671 9235, GHSA-c7hr-j4mj-j2w6). <https://github.com/advisories/GHSA-c7hr-j4mj-j2w6>. Accessed 2026-09-28.

673 Jin, G., Song, L., Shi, X., Scherpelz, J., Lu, S., 2012. Understanding and detecting
674 real-world performance bugs, in: *Proceedings of the 33rd ACM SIGPLAN*
675 *Conference on Programming Language Design and Implementation (PLDI ’12)*,
676 pp. 77–88. doi:10.1145/2254064.2254075.

677 Jones, M.B., Bradley, J., Sakimura, N., 2015. JSON Web Token (JWT). Request for
678 Comments RFC 7519. Internet Engineering Task Force. doi:10.17487/RFC7519.

679 Kalibera, T., Jones, R., 2013. Rigorous benchmarking in reasonable time, in:
680 *Proceedings of the 2013 International Symposium on Memory Management*
681 *(ISMM ’13)*. doi:10.1145/2464157.2464160.

682 Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D.M., Damian, D.,

683 2014. The promises and perils of mining GitHub, in: Proceedings of the 11th
684 Working Conference on Mining Software Repositories (MSR 2014). doi:10.114
685 5/2597073.2597074.

686 Kaltenecker, C., Mühlbauer, S., Grebhahn, A., Siegmund, N., Apel, S., 2023.
687 Performance evolution of configurable software systems: An empirical study.
688 Empirical Software Engineering 28. doi:10.1007/s10664-023-10338-3.

689 Katha, J.F., Prova, T.T., Morol, M.K., Liew, T.H., Nandi, D., 2026. Replica-
690 tion package: a pre-authentication key parse dominates JWT validation in
691 jsonwebtoken. doi:10.5281/zenodo.23011153. Source repository: [https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominat](https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominates-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing-)
692 [es-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing-](https://github.com/Katha-Sikdar/A-Pre-Authentication-Key-Parse-Dominates-JWT-Validation-in-jsonwebtoken-and-Is-Load-Bearing-).
693

694 Kery, M.B., Le Goues, C., Myers, B.A., 2016. Examining programmer practices for
695 locally handling exceptions, in: Proceedings of the 13th International Conference
696 on Mining Software Repositories (MSR '16), pp. 484–487. doi:10.1145/290173
697 9.2903497.

698 Krüger, S., Späth, J., Ali, K., Bodden, E., Mezini, M., 2018. CrySL: An extensi-
699 ble approach to validating the correct usage of cryptographic APIs, in: 32nd
700 European Conference on Object-Oriented Programming (ECOOP 2018), pp.
701 10:1–10:27. doi:10.4230/LIPIcs.ECOOP.2018.10.

702 Laaber, C., Leitner, P., 2018. An evaluation of open-source software microbench-
703 mark suites for continuous performance assessment, in: Proceedings of the
704 15th International Conference on Mining Software Repositories (MSR '18).
705 doi:10.1145/3196398.3196407.

706 Laaber, C., Scheuner, J., Leitner, P., 2019. Software microbenchmarking in the
707 cloud. how bad is it really? Empirical Software Engineering 24. doi:10.1007/s1
708 0664-019-09681-1.

709 Lazar, D., Chen, H., Wang, X., Zeldovich, N., 2014. Why does cryptographic
710 software fail? A case study and open problems, in: Proceedings of the 5th Asia-
711 Pacific Workshop on Systems (APSys '14). doi:10.1145/2637166.2637237.

712 Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2009. Producing wrong
713 data without doing anything obviously wrong!, in: Proceedings of the 14th
714 International Conference on Architectural Support for Programming Languages
715 and Operating Systems (ASPLOS '09). doi:10.1145/1508244.1508275.

716 Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2010. Evaluating the
717 accuracy of Java profilers, in: Proceedings of the 31st ACM SIGPLAN Conference
718 on Programming Language Design and Implementation (PLDI '10), pp. 187–197.
719 doi:10.1145/1806596.1806618.

720 Nadi, S., Krüger, S., Mezini, M., Bodden, E., 2016. Jumping through hoops, in:

721 Proceedings of the 38th International Conference on Software Engineering (ICSE
722 '16). doi:10.1145/2884781.2884790.

723 Naser, A., Sadeghi Lahijani, M., Wu, C., Gavahi, M., Hoang, V.T., Wang, Z., Yuan,
724 X., 2020. Performance evaluation and modeling of cryptographic libraries for
725 MPI communications. arXiv preprint arXiv:2010.06139.

726 Nistor, A., Jiang, T., Tan, L., 2013. Discovering, reporting, and fixing performance
727 bugs, in: Proceedings of the 10th Working Conference on Mining Software
728 Repositories (MSR '13). doi:10.1109/MSR.2013.6624035.

729 Ogasawara, T., Komatsu, H., Nakatani, T., 2006. EDO: Exception-directed
730 optimization in Java. ACM Transactions on Programming Languages and
731 Systems 28. doi:10.1145/1111596.1111598.

732 de Pádua, G.B., Shang, W., 2017. Studying the prevalence of exception handling
733 anti-patterns, in: Proceedings of the 25th International Conference on Program
734 Comprehension (ICPC '17), pp. 328–331. doi:10.1109/ICPC.2017.1.

735 Parravicini, A., Mueller, T., 2021. The cost of speculation: Revisiting overheads in
736 the V8 JavaScript engine, in: 2021 IEEE International Symposium on Workload
737 Characterization (IISWC). doi:10.1109/IISWC53511.2021.00013.

738 Rahaman, S., Xiao, Y., Afrose, S., Shaon, F., Tian, K., Frantz, M., Kantarcioglu,
739 M., Yao, D.D., 2019. CryptoGuard: High precision detection of cryptographic
740 vulnerabilities in massive-sized Java projects, in: Proceedings of the 2019 ACM
741 SIGSAC Conference on Computer and Communications Security (CCS '19), pp.
742 2455–2472. doi:10.1145/3319535.3345659.

743 Selakovic, M., Glaser, T., Pradel, M., 2017. An actionable performance profiler
744 for optimizing the order of evaluations, in: Proceedings of the 26th ACM
745 SIGSOFT International Symposium on Software Testing and Analysis (ISSTA
746 '17). doi:10.1145/3092703.3092716.

747 Selakovic, M., Pradel, M., 2016. Performance issues and optimizations in JavaScript:
748 An empirical study, in: Proceedings of the 38th International Conference on
749 Software Engineering (ICSE '16), pp. 61–72. doi:10.1145/2884781.2884829.

750 Sheffer, Y., Hardt, D., Jones, M.B., 2020. JSON Web Token Best Current Practices.
751 Request for Comments RFC 8725. Internet Engineering Task Force. doi:10.174
752 87/RFC8725.

753 Sikeridis, D., Kampanakis, P., Devetsikiotis, M., 2020. Assessing the overhead of
754 post-quantum cryptography in TLS 1.3 and SSH, in: Proceedings of the 16th
755 International Conference on Emerging Networking EXperiments and Technologies
756 (CoNEXT '20). doi:10.1145/3386367.3431305.

757 Staicu, C.A., Pradel, M., 2018. Freezing the web: A study of ReDoS vulnerabilities
758 in JavaScript-based web servers, in: Proceedings of the 27th USENIX Security
759 Symposium, pp. 361–376.

- 760 Traini, L., Cortellessa, V., Di Pompeo, D., Tucci, M., 2023. Towards effective
761 assessment of steady state performance in Java software: Are we there yet?
762 Empirical Software Engineering 28. doi:10.1007/s10664-022-10247-x.
- 763 Xu, B., Jia, S., Lin, J., Zheng, F., Ma, Y., Liu, L., Gu, X., Song, L., 2023.
764 JWTKey: Automatic cryptographic vulnerability detection in JWT applications,
765 in: Computer Security – ESORICS 2023, Springer. doi:10.1007/978-3-031-5
766 1479-1_14.
- 767 Yang, J., Wang, E., Chen, J., Wang, Q., Zhang, Y., Duan, H., Xie, W., Wang,
768 B., 2026. Token time bomb: Evaluating JWT implementations for vulnerability
769 discovery, in: Proceedings of the Network and Distributed System Security
770 Symposium (NDSS '26).
- 771 Zerouali, A., Mens, T., Robles, G., Gonzalez-Barahona, J.M., 2019. On the
772 relation between outdated Docker containers, severity vulnerabilities, and bugs,
773 in: Proceedings of the 26th IEEE International Conference on Software Analysis,
774 Evolution and Reengineering (SANER '19), pp. 491–501. doi:10.1109/SANER.
775 2019.8668013.
- 776 Zhu, X., She, G., Xue, B., Zhang, Y., Zhang, Y., Zou, X.K., Duan, X., He, P.,
777 Krishnamurthy, A., Lentz, M., Zhuo, D., Mahajan, R., 2023. Dissecting overheads
778 of service mesh sidecars, in: Proceedings of the 2023 ACM Symposium on Cloud
779 Computing (SoCC '23). doi:10.1145/3620678.3624652.